

AlertaRio RS

Sistema de Monitoramento Hidrológico e Climático em Java

Roger da Palma

Universidade Franciscana
Programação Orientada a Objetos

9 de junho de 2026

Objetivo do Projeto

O **AlertaRio RS** é um sistema acadêmico desenvolvido em Java para simular o monitoramento de chuva e nível dos rios em regiões de risco.

O sistema monitora as cidades:

- Porto Alegre
- Caxias do Sul
- Santa Maria

A proposta é registrar medições, comparar valores e gerar alertas de risco de enchente.

Problema que o Sistema Resolve

O problema prático abordado é a dificuldade de acompanhar, de forma organizada, a relação entre chuva intensa e aumento do nível dos rios.

Sem monitoramento, o risco de enchente pode ser identificado tarde demais.

O sistema busca simular uma solução que:

- registra medições periódicas;
- compara valores atuais com anteriores;
- classifica o risco;
- gera alertas preventivos.

Arquitetura do Projeto

O projeto foi organizado em pacotes para separar responsabilidades.

- **app**: classe principal do sistema.
- **model**: entidades principais.
- **service**: regras de negócio.
- **repository**: armazenamento das medições.
- **interfaces**: contratos do sistema.
- **exception**: exceção customizada.
- **enums**: tipos fixos do sistema.

Essa organização facilita a manutenção e demonstra melhor os conceitos de POO.

A classe **Main.java** é responsável por iniciar o sistema.

Ela faz:

- cria as cidades monitoradas;
- cria os sensores de rio e chuva;
- executa a simulação;
- registra as medições;
- chama os serviços de análise;
- trata erros com **try-catch-finally**.

É o ponto de entrada do programa.

A classe **Cidade.java** representa uma cidade monitorada.

Principais atributos:

- nome;
- estado;
- região de risco.

Ela usa encapsulamento com atributos privados, getters e setters.

Também possui sobrecarga de construtores.

A classe **Medicao.java** representa uma medição realizada pelo sistema.

Ela armazena:

- cidade;
- tipo do sensor;
- valor medido;
- data e hora.

Também valida se o valor da medição é negativo. Caso seja, lança uma exceção customizada.

A classe **Alerta.java** representa o alerta gerado após a análise de risco.

Ela possui:

- cidade;
- nível de risco;
- mensagem;
- data e hora.

Exemplo de uso: informar se uma cidade está com risco baixo, moderado, alto ou crítico.

Classe Abstrata Sensor

A classe **Sensor.java** é uma classe abstrata.

Ela representa um sensor genérico do sistema.

Possui atributos comuns:

- identificador;
- cidade;
- última medição.

Não é instanciada diretamente. Ela serve como base para sensores específicos.

As classes **SensorRio.java** e **SensorChuva.java** herdam de **Sensor**.

SensorRio mede o nível do rio em metros.

SensorChuva mede a quantidade de chuva em milímetros.

Essa relação representa:

- SensorRio é um Sensor;
- SensorChuva é um Sensor.

O projeto possui duas interfaces principais.

Monitoravel.java

- define métodos para registrar e obter medições.

GeradorAlerta.java

- define o contrato para classes que geram alertas.

As interfaces ajudam a padronizar comportamentos no sistema.

O projeto utiliza enums para representar valores fixos.

TipoSensor.java

- RIO;
- CHUVA.

NivelRisco.java

- BAIXO;
- MODERADO;
- ALTO;
- CRITICO.

A classe **MedicaoRepository.java** funciona como um armazenamento em memória.

Ela usa um **ArrayList** para guardar as medições.

Principais funções:

- salvar medições;
- listar todas as medições;
- buscar a última medição por cidade e tipo de sensor;
- exibir o histórico final.

A classe **MonitoramentoService.java** concentra a lógica de registro das medições.

Ela faz:

- recebe um sensor e um valor;
- registra a medição;
- salva no repositório;
- compara com a medição anterior.

A comparação pode retornar:

- aumentou;
- diminuiu;
- permaneceu igual.

A classe **AnaliseRiscoService.java** analisa o risco de enchente.

Ela recebe:

- nível do rio;
- quantidade de chuva.

Com base nesses valores, retorna o nível de risco:

- baixo;
- moderado;
- alto;
- crítico.

Também gera a mensagem do alerta.

O projeto possui uma exceção customizada:

MedicaoInvalidaException.java

Ela é usada quando uma medição inválida é informada, como:

- nível do rio negativo;
- quantidade de chuva negativa.

No **Main.java**, o sistema trata erros usando:

- try;
- catch;
- finally.

O polimorfismo aparece quando o sistema cria uma lista do tipo **Sensor**.

Essa lista recebe objetos de tipos diferentes:

- SensorRio;
- SensorChuva.

Mesmo sendo classes diferentes, ambas são tratadas como **Sensor**.

Isso reduz repetição de código e deixa o sistema mais flexível.

Exemplo de Polimorfismo no Código

Trecho importante

```
ArrayList<Sensor> sensores = new ArrayList<Sensor>();  
sensores.add(new SensorRio(...));  
sensores.add(new SensorChuva(...));
```

Depois, o sistema percorre todos os sensores da mesma forma:

Percorrendo a lista

```
for (Sensor sensor : sensores) {  
    monitoramentoService.registrarMedicao(sensor, valor);  
}
```

Como os Valores Funcionam

Os valores são simulados de forma aleatória.

Para sensores de rio:

- o sistema gera valores em metros;
- aproximadamente entre 0,5 m e 6,0 m.

Para sensores de chuva:

- o sistema gera valores em milímetros;
- aproximadamente entre 0 mm e 120 mm.

Esses valores são acadêmicos e servem para demonstrar o funcionamento do sistema.

A análise de risco segue regras simples.

- Rio maior ou igual a 5 m ou chuva maior ou igual a 100 mm: **CRÍTICO**.
- Rio maior ou igual a 4 m ou chuva maior ou igual a 80 mm: **ALTO**.
- Rio maior ou igual a 3 m ou chuva maior ou igual a 50 mm: **MODERADO**.
- Abaixo desses valores: **BAIXO**.

O funcionamento geral do sistema segue este fluxo:

- 1 O sistema cria as cidades.
- 2 Cria sensores de rio e chuva.
- 3 Gera valores simulados.
- 4 Registra as medições.
- 5 Salva no repositório.
- 6 Compara com a medição anterior.
- 7 Analisa o risco.
- 8 Gera e exibe alertas.

O projeto atende aos principais requisitos da disciplina:

- classes, objetos e atributos;
- construtores;
- encapsulamento;
- herança;
- polimorfismo;
- classe abstrata;
- interfaces;
- tratamento de exceções.

O **AlertaRio RS** demonstra como a Programação Orientada a Objetos pode ser aplicada em um problema real.

O projeto modela cidades, sensores, medições, alertas e análise de risco de forma organizada.

Mesmo sendo uma simulação acadêmica, o sistema pode ser expandido futuramente para usar dados reais de sensores ou APIs climáticas.

Obrigado!